Session 6 · Deck 6.A1

Aiyagari I: The Household, From Bellman to Euler

Zhigang Feng

Thursday, October 29, 2026

© Zhigang Feng • Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 6.A1 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5 ✓	S6	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — Aiyagari: 6.A1 the household, from Bellman to Euler · 6.A2 the population and the price.
Block B: 6.B1 Krusell–Smith · 6.B2 transitions and the life cycle.

© Zhigeng Fang
Redistribution prohibited without permission

Roadmap: Seven Parts

Block A computes the economy 2.A2 defined. This deck is its innermost step: what a household does at given prices — through its value function, then through its Euler equation.

Part	Topic	The question it answers
1	The economy and the plan	What are we solving, and which earlier tool does each step use?
2	The function we are after	What is the unknown, and why must it live on a grid?
3	Value function iteration	Iterating on v: why it converges, and what it costs.
4	Between the nodes	How does a' leave the grid: interpolate v, then maximize?
5	The Euler route	How do time iteration and EGM iterate on the policy instead?
6	Projection	Can 140 coefficients replace 1,400 node values? What does the kink cost?
7	Compare and grade	Every solver on one problem, and a reference that decides.

© Zhigang Feng · Redistribution prohibited without permission

Idea

One household problem, solved by every route on the same grid at the same price, $r = 2\%$. Every number in this deck is reproduced by Lab 6.

2.A2's economy at the calibration of Lab 6. Annual; every number on this frame is an input.

Households. A continuum, each maximizing $\mathbb{E}_0 \sum_t \beta^t u(c_t)$ with $u(c) = c^{1-\sigma}/(1-\sigma)$, subject to

$$c + a' = (1+r)\,a + w\varepsilon, \qquad a' \geq \underline{a} = 0.$$

Labour productivity ε follows a Markov chain Π with mean one.

The firm. $Y = K^{\alpha} L^{1-\alpha}$ with $L = 1$, so

$$r = \alpha K^{\alpha-1} - \delta, \qquad w = (1-\alpha) K^{\alpha}.$$

α	0.33	capital share
β	0.96	discount factor
δ	0.08	depreciation
σ	2	CRRA
ρ_ε	0.90	persistence of $\log \varepsilon$
s.d.	0.20	innovation (0.46 unconditional)
ε	7 states	Rouwenhorst: $0.29, \ldots, 2.77$

Example: the asset grid

$a_i = 150\,(i/199)^{2.5}$, $i = 0, \ldots, 199$: the first steps are 0.0003, 0.0013, 0.0027, the last 1.88; 27 of the 200 nodes lie below $a = 1$. Dense where the limit binds (3.A2, "Decision 2").

© Zhigang Feng Redistribution prohibited without permission

Ledger `core.params`, `core.earnings`, `core.grid`. **Rouwenhorst as in 4.B2.**

2.A2's stationary equilibrium in this deck's notation: prices are numbers, the distribution sits still.

Definition

A **stationary recursive competitive equilibrium** is prices (r, w), a value $v(a, \varepsilon)$, policies $c(a, \varepsilon)$, $a'(a, \varepsilon)$, a distribution λ over (a, ε) and capital K such that:

1. **Households:** given (r, w), v solves the Bellman equation and (c, a') are its policies.
2. **Firm:** $r = \alpha K^{\alpha-1} - \delta$ and $w = (1-\alpha)K^{\alpha}$.
3. **Labour:** $L = \int \varepsilon \, d\lambda = 1$.
4. **Capital:** $K = \int a \, d\lambda$: what households hold is what the firm rents.
5. **Stationarity:** pushing λ through a' and Π returns λ: $\lambda(\mathcal{B}, \varepsilon') = \sum_{\varepsilon} \Pi(\varepsilon, \varepsilon') \, \lambda\big(\{a : a'(a, \varepsilon) \in \mathcal{B}\}, \varepsilon\big)$ for every set $\mathcal{B}$.

Redistribution prohibited without permission

Idea

Each household takes prices as given; the prices must be the ones its population implies. Conditions 4 and 5 are solved together.

© Zhuoqing Feng

Where each condition is solved

1: this deck. 5: 6.A2, Parts 1–4. 4: 6.A2, Part 5, by bisection on r. 3 holds by construction, since ε has mean one.

2.A2, "Stationary Recursive Competitive Equilibrium"; Aiyagari, *QJE* (1994).

Three Problems, Nested

Fix a price, solve the household, find the population it produces, compare with the firm. Each step is a fixed point inside the next.

0. Fix $r \in (-\delta,\ 1/\beta - 1)$; the firm then gives $w(r)$.

1. A household (this deck). Solve the Bellman equation at (r, w): the value v_r, the saving policy a'_r, consumption c_r.

2. A population (6.A2). a'_r and Π move households across (a, ε); find the invariant distribution λ_r.

3. A price (6.A2). Excess supply $d(r) = \int a \, d\lambda_r - K^d(r)$; adjust r until $d(r) = 0$.

Idea

Each layer is a fixed point containing the next: one evaluation of $d(r)$ costs a whole dynamic program and a whole distribution. Step 1 runs about twenty times, so its speed is the budget.

Pitfall

$d(r)$ is household supply minus firm demand, and both move with r. Writing supply on both sides makes $d \equiv 0$: easy to do, and invisible in the output.

© Zhigang Feng
Redistribution prohibited without permission

The equilibrium is 2.A2's stationary recursive competitive equilibrium. 6.A2's bisection takes 20 steps, ledger `core.equilibrium`.

1 · The Toolkit, and Where Each Piece Is Used

Nothing in Block A is new technique. Every step uses a tool from Sessions 1–5 — and two tools stay on the shelf, on purpose.

Step	Tool	Taught in	Why it is used here
income process	Rouwenhorst, 7 states	4.B2	$\rho_\varepsilon = 0.9$: Tauchen's states stick
asset grid	power grid, dense at $\underline{a}$	3.A2, Decision 2	the kink is at the limit
the function	VFI, Howard steps	1.B2, 3.A2, 4.B3	works for any problem
between nodes	linear, PCHIP, spline	3.A2, 4.B1	a' is a continuous choice
the choice	golden section	4.A1	one peak, no derivative
the policy	time iteration; EGM	3.A3; 4.B3	no maximization at all
the policy, again	Chebyshev collocation	5.B2, 5.B4	does the kink rule it out? Part 6
accuracy	Euler residual between nodes	3.A3	"stopped changing" is not a grade
the population	a Markov chain on the grid	4.B2; 6.A2	λ from a' and Π
the price	bisection, with a sign check	4.A1; 6.A2	$d(r)$ is only piecewise smooth
on the shelf	perturbation	5.A	the kink of Part 2
on the shelf	Newton on the price	4.A1	no reliable derivative (6.A2)

© Zhigang Feng Redistribution prohibited without permission

The Unknown Is a Function: the Bellman Equation

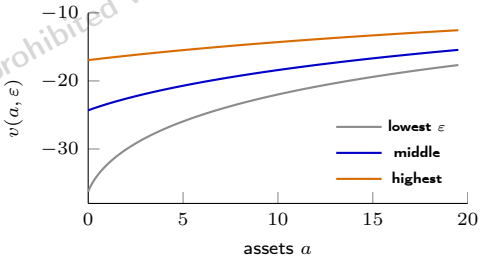
What the household solves, written once. The unknown is not a number but a function of two variables — on our grid, 1,400 numbers.

$$v(a,\varepsilon) = \max_{\underline{a} \le a' < \mathrm{coh}} \Big\{ u(\underbrace{\mathrm{coh} - a'}_{c}) + \beta \sum_{\varepsilon'} \Pi(\varepsilon,\varepsilon')\, v(a',\varepsilon') \Big\},$$

with cash on hand $\mathrm{coh} = (1+r)a + w\varepsilon$. It delivers three functions of the state: the **value** $v(a,\varepsilon)$, the **saving policy** $a'(a,\varepsilon)$ and **consumption** $c = \mathrm{coh} - a'$. On the grid: 200 asset nodes $\times$ 7 productivity levels $= 1{,}400$ numbers each.

Idea

Every method in this deck is a rule for updating those 1,400 numbers until they stop moving — the value (Parts 3–4) or the policy (Part 5).



At $r = 2\%$: increasing and concave in a, shifted up by ε, steepest at the limit.

Ledger `household_methods.fig_value_2pct`: **VFI with linear interpolation and Howard steps (Parts 3–4)**.

© Zhigeng Feng
Redistribution prohibited without permission

The Kink, Why a Grid, and Two Ways to Iterate

The borrowing limit turns the Euler equation into an inequality and puts a kink in the policy. That kink decides the method.

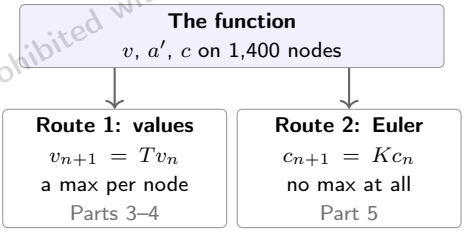
$$u'(c) \;\geq\; \beta(1+r)\,\mathbb{E}\big[u'(c') \mid \varepsilon\big], \qquad \text{equality if } a' > \underline{a}.$$

Below a threshold $\bar{a}(\varepsilon)$ the household would like to borrow, so $a' = \underline{a}$ and $c = \text{coh}$. At $r = 2\%$: $\bar{a} = 0.033$ for the lowest ε, then 0.026 and 0.001; never for the upper four. At $\bar{a}$ the policy has a **kink**.

Why not Session 5's tools

A Taylor series around one point (5.A) cannot carry a kink across a population, and a global polynomial (5.B) rings at it (4.B1, Gibbs). Hence a grid, straight lines between nodes, dense where the kink is. Part 6 measures the ringing.

Ledger `core.kink_2pct`: **the threshold is where the endogenous grid of Part 5 starts.**



1.B2's "Four Design Choices" with one choice changed: same model, same grid — only *which equation* we iterate on differs.

© Zhigang Feng Redistribution prohibited without permission

Value Function Iteration: the Operator, and Why It Converges

Apply the right-hand side of the Bellman equation to a guess, and repeat. The map is a contraction, so the change shrinks by the factor β every sweep.

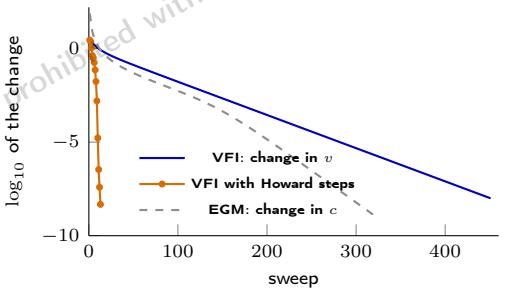
$$(Tv)(a_i, \varepsilon_q) = \max_{a'} \left\{ u(\mathrm{coh}_{iq} - a') + \beta \sum_{q'} \Pi_{qq'}\, v(a', \varepsilon_{q'}) \right\}.$$

Why it converges (1.B2, Blackwell):
$\|Tv - Tw\| \le \beta \, \|v - w\|$. From any start the sweeps $v_{n+1} = Tv_n$ approach one fixed point, and the change $\|v_{n+1} - v_n\|$ falls by β per sweep.

How long: from $v_0 = 0$ the change starts near one, so a tolerance ϵ takes about $\ln \epsilon / \ln \beta$ sweeps — 1.B2's rule.

Example

$\beta = 0.96$, $\epsilon = 10^{-8}$: $\ln 10^{-8} / \ln 0.96 = 451$ sweeps predicted, **451** measured.



The VFI line falls by $\log_{10} 0.96$ per sweep; with Howard steps, 13 maximizations suffice.

Ledger `household_methods.contraction`, `fig_convergence`; $r = 2\%$.

© Yaorang Feng · Redistribution prohibited without permission

3.A2's ten lines with one line added: the expectation over tomorrow's productivity, which is a matrix product.

Input: nodes $a_1 < \cdots < a_{200}$, levels $\varepsilon_1, \ldots, \varepsilon_7$, Π; prices r, w; tolerance ϵ.

1: $\text{coh}_{iq} \leftarrow (1+r)a_i + w\varepsilon_q$
2: $U_{ijq} \leftarrow u(\text{coh}_{iq} - a_j)$ if positive, else -10^{10} ▷ built once
3: $v \leftarrow 0$
4: **repeat**
5: $\quad EV_{jq} \leftarrow \sum_{q'} \Pi_{qq'} v_{jq'}$ ▷ expectation
6: $\quad v^{\text{new}}_{iq} \leftarrow \max_j \{U_{ijq} + \beta EV_{jq}\}$; $j^*_{iq} \leftarrow$ the best j
7: $\quad d \leftarrow \max |v^{\text{new}} - v|$; $v \leftarrow v^{\text{new}}$
8: **until** $d < \epsilon$
9: $a'(a_i, \varepsilon_q) \leftarrow a_{j^*_{iq}}$ ▷ never leaves the grid

The trick worth naming

U does not depend on v: it is built once, $200 \times 200 \times 7 = 280{,}000$ entries, and reused every sweep. Infeasible choices ($c \leq 0$) get -10^{10} instead of a test inside the loop.

What one sweep costs

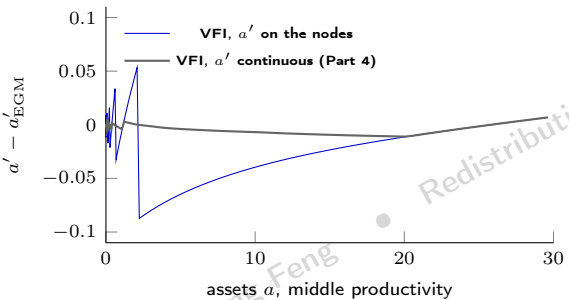
$280{,}000$ additions and comparisons. EGM (Part 5) does $1{,}400$ inversions.

Ledger `core.household_2pct`: **grid-search evaluations per sweep against EGM's inversions.**

© Zhigang Feng
Redistribution prohibited without permission

What a' on the Nodes Costs, and What It Gets Wrong

Forcing a' onto the nodes makes the policy a staircase. Its errors do not cancel: they lean one way, and everything computed from the policy inherits them.



Measured, $r = 2\%$

451 sweeps, 0.17 s. Below $a = 30$ the household saves less than EGM's policy at 76% of the nodes, by up to 0.087 (at $a = 2.24$, where a grid step is 0.15). Mean Euler residual $10^{-1.87}$. Capital supplied: 4.34 instead of 4.66, **7% short**.

Cheap per sweep and robust, with a biased answer. More sweeps cannot fix it; a continuous choice can (Part 4).

Ledger `household_methods.rows` **(**`vfi_grid`**) and** `fig_policy_gap_mid`**. Capital supplied: the policy's invariant distribution, computed as in 6.A2.**

© Zhigang Feng
Redistribution prohibited without permission

3 · Making VFI Fast: Howard, Monotonicity, Concavity, a Good Start

The standard accelerations. Each uses a property of this problem; none changes the answer.

- **Howard (4.B3).** After each maximization, update v n_H times with the policy held fixed — no max. With $n_H = 50$: **13** maximizations instead of 451, 0.16 s instead of 4.1 s, the same policy.
- **Monotonicity (4.B3).** $a'(a, \varepsilon)$ rises with a, so the search at node $i+1$ can start at node i's choice.
- **Concavity (4.B3).** The objective has one peak: stop when it falls, or shrink a bracket — golden section, Part 4.
- **A good start.** At 6.A2's equilibrium price, starting from the value found at $r = 2\%$ takes **7** maximizations instead of 13. Inside a price loop that saving recurs at every price.
- **Multigrid.** Solve on a coarse grid, interpolate, refine. Not run here.

Idea: Howard in one line

Maximizing is the expensive step, and the policy settles long before the values do (1.B2's hook). So search rarely, and commit often.

Example

The orange dots of two frames ago: 13 maximizations, each followed by 50 cheap updates of v, reach the tolerance that plain VFI needs 451 maximizations for.

© Zhigang Feng · Redistribution prohibited without permission

Ledger `household_methods.rows` (`vfi_linear`, `vfi_linear_howard`) **and** `vfi_warm_start_rstar`.

4 · Let a' Leave the Grid: Interpolate v, Then Maximize

The grid stores v only at nodes; between them, an interpolant $\hat{V}$ (4.B1). With $\hat{V}$ concave the objective has one peak, which golden section (4.A1) finds.

1: **repeat**
2: $EV_{jq} \leftarrow \sum_{q'} \Pi_{qq'} v_{jq'}$ ▷ at the nodes
3: for each q: $\hat{V}_q \leftarrow$ interpolant through $(a_j, EV_{jq})_j$ ▷ 4.B1
4: $a'_{iq} \leftarrow \arg\max_{x \in [\underline{a},\, \mathrm{coh}_{iq}]} u(\mathrm{coh}_{iq} - x) + \beta \hat{V}_q(x)$
5: golden section, then compare with $x = \underline{a}$ ▷ 4.A1
6: $v_{iq}^{\mathsf{new}} \leftarrow$ that maximum; $d \leftarrow \max |v^{\mathsf{new}} - v|$
7: $v \leftarrow v^{\mathsf{new}}$; Howard: n_H times $v_{iq} \leftarrow u(\mathrm{coh}_{iq} - a'_{iq}) + \beta \hat{V}_q(a'_{iq})$
8: **until** $d < \epsilon$

Why it works

Linear $\hat{V}$ keeps concavity (4.B1), so the objective has one peak and golden section cannot be fooled. All 1,400 brackets shrink at once: 55 evaluations each, 77,000 per sweep.

One step further

Replace $\hat{V}$ by a neural network and Line 4 is Session 8's deep value iteration.

Ledger `household_methods.rows`: **50 golden-section steps, plus two to start and three at the end.**

© Zhigang Feng
Redistribution prohibited without permission

Which Interpolant? Linear, PCHIP, Spline

The same Howard steps with three interpolants. Away from the kink v is smooth, and a smooth $\hat{V}$ buys two orders of magnitude in the policy.

$\hat{V}$	guarantees (4.B1)	maximizations	seconds	$\max \lvert a' - a'_{\text{EGM}} \rvert$	mean Euler residual
linear	monotone, concave	13	0.16	2.6×10^{-2}	$10^{-3.28}$
PCHIP	monotone	12	0.86	4.7×10^{-4}	$10^{-5.47}$
cubic spline	none	12	0.75	4.8×10^{-4}	$10^{-6.02}$

Why linear loses

A linear $\hat{V}$ has a kink at every node, where its slope jumps. The first-order condition $u'(c) = \beta \hat{V}'(a')$ then holds either inside a cell or at a node: the policy inherits small steps. PCHIP and splines have a continuous slope.

Pitfall

The spline's guarantee is *none*. It behaved here because v is smooth where the households are; 4.B1 showed kinked data on which it overshoots. Measure, do not assume.

Ledger `household_methods.rows`; `PchipInterpolator` **and** `CubicSpline` **from** `scipy.interpolate`.

© Zhong Jiang University · Redistribution prohibited without permission

Why the Euler Equation Is Enough

In a concave problem the first-order condition, with the limit, pins down the optimum. So we can iterate on the policy and never maximize.

The unknown becomes the policy $c(a, \varepsilon)$, and the Euler inequality is a functional equation in it (3.A3). Given tomorrow's policy c_n, today's c at a node solves

$$u'(c) = \beta(1+r) \sum_{q'} \Pi_{qq'}\, u'\big(c_n(\mathrm{coh} - c,\ \varepsilon_{q'})\big),$$

or $c = \mathrm{coh} - \underline{a}$ if that solution would leave $a' < \underline{a}$. Call this map K: $c_{n+1} = K c_n$.

Idea

The maximization is replaced by its first-order condition. One equation per node, and no search.

	Values	**Euler**
unknown	$v(a, \varepsilon)$	$c(a, \varepsilon)$
iteration	$v_{n+1} = T v_n$	$c_{n+1} = K c_n$
converges by	contraction, rate β (1.B2)	monotonicity (3.A3)
needs	a bounded return	concavity, differentiability
strength	any problem	no maximization
weakness	a max at every node	the FOC must suffice

The monotone-operator argument for K is 3.A3's ("Monotonicity Instead of Contraction").

© Zhaonan Feng
Redistribution prohibited without permission

Time Iteration: Solve the Euler Equation at Every Node

3.A3's time iteration with a borrowing limit: at each node, a root-find for today's c, with tomorrow's policy read between nodes.

1: $c \leftarrow \text{coh} - \underline{a}$ ▷ start: eat everything
2: **repeat**
3: $g(x) \equiv u'(x) - \beta(1+r)\sum_{q'} \Pi_{qq'}\, u'\big(c(\text{coh} - x, \varepsilon_{q'})\big)$
4: if $g(\text{coh} - \underline{a}) \geq 0$: $c^{\text{new}} \leftarrow \text{coh} - \underline{a}$ ▷ the limit binds
5: else bisect g on $(0, \text{coh} - \underline{a})$, 60 halvings ▷ 4.A1
6: $d \leftarrow \max|c^{\text{new}} - c|$; $c \leftarrow c^{\text{new}}$
7: **until** $d < 10^{-9}$

Tomorrow's c is read linearly between nodes. g falls as x rises, so the bisection always has a bracket; all 1,400 nodes are bisected at once.

Measured, $r = 2\%$

323 sweeps, the same count as EGM: the same operator, discretized differently. But a sweep costs 61 evaluations of g per node, each with 7 interpolations: 5.7 s, against EGM's 0.05 s.

It agrees with EGM to 1.5×10^{-5} in a'. Its cost is the root-find, and EGM removes it.

© Zhigang Feng · Redistribution prohibited without permission

Ledger `household_methods.rows (time_iteration), agreement.`

EGM: Put the Grid on Tomorrow's Assets, and Invert

Turn the root-find around: fix a' on a node and solve for today. The Euler equation then gives c in closed form (4.B3).

1: $\mathbb{E}u'_{jq} \leftarrow \sum_{q'} \Pi_{qq'}\, c_n(a_j, \varepsilon_{q'})^{-\sigma}$ ▷ $a' = a_j$: no interpolation
2: $\tilde{c}_{jq} \leftarrow \left(\beta(1+r)\,\mathbb{E}u'_{jq}\right)^{-1/\sigma}$ ▷ invert u'
3: $\tilde{a}_{jq} \leftarrow (\tilde{c}_{jq} + a_j - w\varepsilon_q)/(1+r)$ ▷ who chose $a' = a_j$
4: $c(a_i, \varepsilon_q) \leftarrow$ linear interpolation of $(\tilde{a}_{jq}, \tilde{c}_{jq})_j$ at a_i
5: where $a_i < \tilde{a}_{1q}$: $c \leftarrow \mathrm{coh}_{iq} - \underline{a}$ ▷ the limit binds
6: $a' \leftarrow \mathrm{coh} - c$

One application is one EGM step; `solve_household` repeats it until $\max|c_{n+1} - c_n| < 10^{-9}$.

Idea

VFI searches over a' for each a. EGM picks a' and asks which a chose it — the endogenous grid $\tilde{a}$. No search, no root-find: 1,400 inversions per sweep.

Example

$(\tilde{a}, \tilde{c})$ are points of today's policy at assets we did not choose; Line 4 carries them to the nodes we did.

© Zhigang Feng
Redistribution prohibited without permission

Carroll, "The method of endogenous gridpoints", *Economics Letters* (2006); 4.B3, "EGM in Four Steps".

Where the Constraint Goes, and One Sweep by Hand

The limit enters as a comparison, not an equation. And from "eat everything", a single sweep already shows who saves.

Line 5. $\tilde{a}_{1q}$ is the asset level at which a household chooses exactly $a' = \underline{a}$ with the Euler equation holding. Anyone poorer would like to borrow, so for $a < \tilde{a}_{1q}$: $a' = \underline{a}$ and $c = \mathrm{coh}$. The kink of Part 2 is this one comparison.

Pitfall: interpolating back

Line 4 is linear, which keeps c increasing. Above the largest $\tilde{a}$, `np.interp` holds c flat: harmless while nobody lives there, wrong if someone did. The grid's top must lie beyond the richest household (6.A2 checks it).

One sweep from $c = \mathrm{coh}$

After one EGM step the lowest-productivity household saves at 96.5% of the nodes; the middle and the highest at all of them.

The whole solve

At $r = 2\%$: 323 sweeps to $\max|c_{n+1} - c_n| < 10^{-9}$, 0.05 s. At 6.A2's equilibrium price: 356 sweeps from scratch, **142** from the previous price's policy.

© Zhigang Feng — Do not distribution prohibited without permission

Ledger `core.one_sweep`, `core.household_2pct`, `core.warm_start`.

6 · Projection: 140 Coefficients Instead of 1,400 Node Values

Session 5's projection, on this household: a Chebyshev polynomial in assets for each productivity level, and the Euler equation imposed at as many points as there are coefficients.

$$\hat{c}(a,\varepsilon_q)=\sum_{k=0}^{n-1}\theta_{kq}\,T_k\big(x(a)\big),\quad x(a)=2\Big(\tfrac{a-\underline{a}}{a_{\max}-\underline{a}}\Big)^{1/2.5}-1.$$

Step	Here	From
basis	$T_k=2xT_{k-1}-T_{k-2}$, on $[-1,1]$	5.B2
stretch	$x(a)$ bends like the grid	3.A2
unknowns	$n=20$ per level, $\times 7$ levels: 140	5.B1
nodes	the 20 zeros of T_{20}, in assets	5.B2
residual	Euler with the limit; c' from $\hat{c}$	Part 5

Idea

On the grid, c is 1,400 numbers joined by straight lines. Here it is 140 numbers defining a smooth function: tomorrow's c is read from it.

Example: Part 5 was already one

Time iteration is a projection too: tent functions on the 200 nodes (5.B3), collocation at the nodes. Part 6 changes only the basis.

© Zhigang Feng Redistribution prohibited without permission

5.B4: the same five steps, on the growth model.

Solving for the Coefficients: Iterate on Them, as Time Iteration Does

Make the Euler equation hold at the nodes, refit, repeat. Newton on all 140 at once fails at the limit.

1: $\theta \leftarrow$ the fit of $c = ra + w\varepsilon$ ▷ consume income
2: **repeat**
3: at each node a_i: bisect the Euler eq., c' from $\hat{c}$ ▷ Part 5
4: where the limit binds: $c_i \leftarrow \mathrm{coh}_i - \underline{a}$
5: $\theta \leftarrow V^{-1}c$, $V_{ik} = T_k(z_i)$ ▷ now $\hat{c}(a_i) = c_i$
6: **until** $\max_i |c_i - \hat{c}(a_i)| < 10^{-10}$

V is 20×20 and fixed. Each sweep is Part 5's operator K, then a refit. A polynomial refit, unlike a straight line, need not preserve order: convergence is measured, not guaranteed (3.A3).

Measured, $n = 20$, $r = 2\%$

365 sweeps, 4.6 s: time iteration's operator (323 sweeps), at 140 nodes instead of 1,400, but each $\hat{c}$ sums 20 terms.

Pitfall: Newton, as in 5.B4

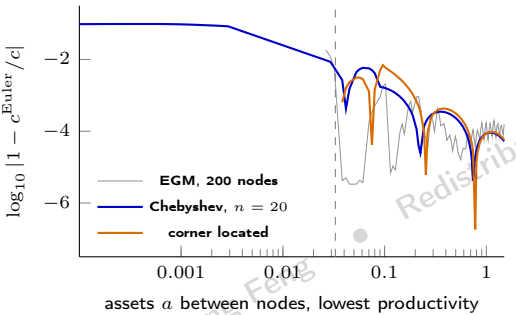
All 140 equations at once (the limit by Fischer–Burmeister, 5.A1), same start: it stops after 1,294 evaluations at a residual of 0.49. Fischer–Burmeister has a corner where both arguments vanish.

© Zhigang Feng · redistribution prohibited without permission

Ledger `projection.rows`, `projection.newton_cold` (`scipy.optimize.root`, `hybr`).

The Kink, Measured: Where the Polynomial Rings, and Locating the Corner

Part 2 predicted it: a polynomial cannot bend at $\bar{a}$. Find the corner in closed form, approximate only the smooth piece, and most of the worst error goes.



The ringing. Worst residual $10^{-1.01}$, at $a = 0.0006$, left of the kink at $\bar{a} = 0.033$ (dashed): Gibbs (5.B2). More terms do not help: $10^{-1.02}$ at $n = 40$.

The fix. Each sweep, ask who chooses $a' = \underline{a}$ exactly: invert the Euler equation there, $\bar{a}_q = (\tilde{c}_q + \underline{a} - w\varepsilon_q)/(1 + r)$ — EGM's Line 5. Below $\bar{a}_q$, $c = \text{coh} - \underline{a}$ exactly; the polynomial covers $[\bar{a}_q, a_{\max}]$ only.

Measured

Worst $10^{-1.01} \to 10^{-2.16}$. What is left sits at $a = 0.096$: from there a' lands on the same corner next period ($a = 0.096$). The kink echoes one step back.

© Zhigang Feng
Redistribution prohibited without permission

Ledger `projection.fig_euler_low`, `projection.rows`, `projection.echo_of_corner`.

Every Solver, One Policy

Same household, grid and price ($r = 2\%$): agreement with EGM, and the Euler residual between nodes.

Solver	sweeps	seconds	work per sweep	$\max \lvert a' - a'_{\text{EGM}} \rvert$	Euler, mean	K^s
VFI, a' on the nodes	451	0.17	280,000 comparisons	0.80	$10^{-1.87}$	4.34
VFI, linear $\hat{V}$	451	4.07	77,000 evaluations	0.026	$10^{-3.28}$	4.63
the same, Howard	13	0.16	77,000 + Howard	0.026	$10^{-3.28}$	4.63
VFI, PCHIP $\hat{V}$, Howard	12	0.86	77,000 + Howard	0.0005	$10^{-5.47}$	4.65
VFI, spline $\hat{V}$, Howard	12	0.75	77,000 + Howard	0.0005	$10^{-6.02}$	4.65
time iteration	323	5.68	85,400 evaluations	1.5×10^{-5}	$10^{-5.63}$	4.66
EGM	323	0.05	1,400 inversions	—	$10^{-5.54}$	4.66
Chebyshev, $n = 20$ (Part 6)	365	4.60	8,540 evaluations	0.0014	$10^{-6.97}$	4.65
corner located	365	4.65	8,540 evaluations	0.0013	$10^{-7.08}$	4.65

The continuous routes reach the same policy. The interpolant sets the accuracy, the route the cost — two orders of magnitude apart.

When EGM does not apply (4.B3)

Discrete choices, non-concavity, a second endogenous state: then only the value route is left (6.B1).

Ledger `household_methods.rows`, `projection.rows` **(Chebyshev: c from the polynomial).** K^s**: from each policy's** λ **(6.A2).**

© Zhiyang Feng
Redistribution prohibited without permission

Which Policy Is Right? A Reference on 16,000 Nodes

Agreeing with EGM is not accuracy. A 16,000-node EGM run is the referee, and it moves the verdict.

Policy	size	$\max \lvert c - c_{\text{ref}} \rvert$	weighted	K^s
reference (16,000)	112,000	—	—	4.6477
EGM, 200 nodes	1,400	5.3×10^{-4}	3.5×10^{-4}	4.6559
Chebyshev $n = 10$	70	4.9×10^{-3}	4.5×10^{-4}	4.6463
Chebyshev $n = 20$	140	1.4×10^{-3}	2.0×10^{-5}	4.6476
Chebyshev $n = 40$	280	1.6×10^{-3}	2.7×10^{-5}	4.6480
corner located	140	1.4×10^{-3}	4.7×10^{-5}	4.6483
no stretch	140	4.9×10^{-2}	4.2×10^{-3}	4.6052

Weighted: by where households are. 4,000 vs 16,000 nodes: 8.0×10^{-6}.

Example: capital

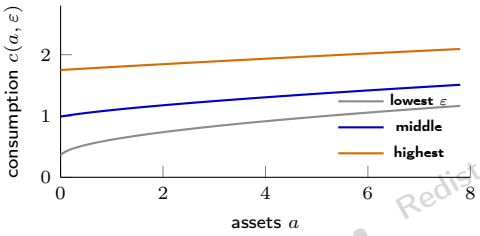
Reference 4.6477. Chebyshev 4.6476 and spline VFI 4.6477 land on it. EGM (4.6559) and time iteration (4.6557) agree with each other, not with it: the error of 200 nodes and straight lines.

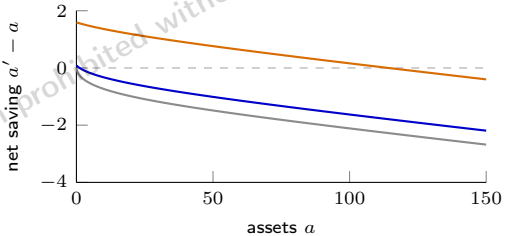
Where households live, 140 coefficients are 18 times closer than 1,400 node values. Near the kink EGM still wins, and it is a hundred times faster. Without the stretch, the worst error is 36 times larger.

© Zhigang Feng Reproduction prohibited without permission

Ledger `projection.reference`, `projection.egm`, `projection.rows`, `projection.no_stretch_n20`; `household_methods.rows`.

Consumption is steep near the limit and flattens with wealth. Net saving changes sign: the poor save, the rich dissave.





Redistribution prohibited without permission

Example: where saving stops

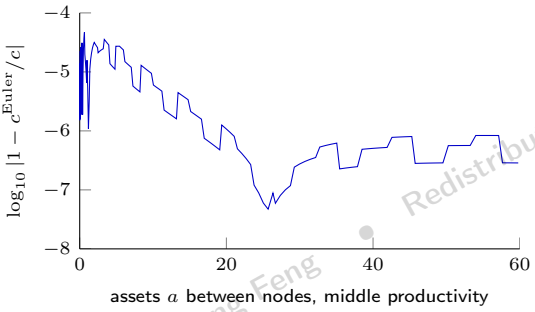
At $r = 2\%$ the middle productivity level saves until $a = 1.15$, the highest until $a = 114$; beyond, assets fall. Below $1/\beta - 1$ wealth has a ceiling — why 2.A2's supply curve is finite there.

© Zhiyang Jia

Ledger `core.fig_consumption_2pct`, `fig_net_saving_2pct`, `target_wealth_2pct`; **EGM policy**.

Accuracy Where We Did Not Fit

"The policy stopped changing" is a statement about the iteration. The test of the answer is the Euler equation at points between the nodes (3.A3).



© Zhigang Feng
Redistribution prohibited without permission

Measured, EGM, $r = 2\%$

Three points in every grid cell; mean residual $10^{-5.54}$. The worst, $10^{-1.73}$, is at $a = 0.026$ (lowest level), in the cell holding the kink at 0.033: a straight line cuts the corner.

What moves it

More nodes. At 6.A2's equilibrium the mass-weighted mean falls from $10^{-4.38}$ (100 nodes) to $10^{-6.22}$ (800); the worst point, at the kink, stays near 10^{-2} until 800 nodes.

Ledger `core.euler_2pct`, `fig_euler_mid_2pct`, `core.refinement`.

How It Works (1): Value Function Iteration, With Golden Section

```python
def vfi_interp(r_net, w, transfer, grid, e, Pi, p: Params, interp="linear", n_howard=0,
               v_init=None, tol=1e-8, max_iter=10000, n_golden=50):
    """VFI with v between the nodes from an interpolant and a' anywhere in
    [a_min, cash on hand): one golden-section search per node and sweep."""
    coh = cash_on_hand(r_net, w, transfer, grid, e)
    top = np.minimum(coh - 1e-8, grid[-1])                    # the largest feasible a'
    v = np.zeros_like(coh) if v_init is None else v_init.copy()
    ap, history, evals = np.empty_like(coh), [], 0
    for it in range(1, max_iter + 1):
        EV, v_new = v @ Pi.T, np.empty_like(v)                # E[v(a', e') | e] at nodes
        for q in range(e.size):
            vhat = interpolant(interp, grid, EV[:, q])        # ... and between them
            obj = lambda x: u(coh[:, q] - x, p.sigma) + p.beta * vhat(x)
            ap[:, q], v_new[:, q], n = golden_max(obj, p.a_min, top[:, q], n_golden)
            evals += n * grid.size
        history.append(float(np.abs(v_new - v).max()))
        v = v_new
        if history[-1] < tol:
            break
        v = howard(v, ap, coh, grid, Pi, p, interp, n_howard)   # optional
        evals += n_howard * v.size
    return dict(v=v, ap=ap.copy(), c=coh - ap, sweeps=it, history=history,
                evaluations=evals)
```

© Zhiwei Peng · Redistribution prohibited without permission

`interpolant` returns `np.interp`, `PchipInterpolator` or `CubicSpline`; `golden_max` and `howard` are on the next frame.

Lab 6, Part A2b runs it.

How It Works (2): Golden Section on All Nodes at Once, and Howard Steps

```python
def golden_max(obj, lo, hi, n_steps=50):
    """Maximise obj on [lo, hi] at every node: golden section (4.A1), then both ends."""
    # PHI = (np.sqrt(5.0) - 1.0) / 2.0  # 0.618..., the golden ratio; set at module top
    a, b = np.broadcast_arrays(np.asarray(lo, float), np.asarray(hi, float))
    x1, x2 = b - PHI * (b - a), a + PHI * (b - a)
    f1, f2 = obj(x1), obj(x2)
    for _ in range(n_steps):
        left = f1 > f2                                          # the peak is in [a, x2]
        a, b = np.where(left, a, x1), np.where(left, x2, b)
        x_new = np.where(left, b - PHI * (b - a), a + PHI * (b - a))
        f_new = obj(x_new)
        x1, x2 = np.where(left, x_new, x2), np.where(left, x1, x_new)
        f1, f2 = np.where(left, f_new, f2), np.where(left, f1, f_new)
    x = 0.5 * (a + b)
    cands = np.stack([x, np.broadcast_to(lo, x.shape), np.broadcast_to(hi, x.shape)])
    vals = np.stack([obj(x), obj(cands[1]), obj(cands[2])])
    best = vals.argmax(axis=0)                                  # interior, a_min or top
    pick = np.arange(x.size)
    return cands[best, pick], vals[best, pick], n_steps + 5
```

```python
def howard(v, ap, coh, grid, Pi, p: Params, interp, n_steps):
    """Policy evaluation: keep a' fixed and update v n_steps times; no maximisation."""
    for _ in range(n_steps):
        EV = v @ Pi.T
        for q in range(Pi.shape[0]):
            vhat = interpolant(interp, grid, EV[:, q])
            v[:, q] = u(coh[:, q] - ap[:, q], p.sigma) + p.beta * vhat(ap[:, q])
    return v
```

© Chunguang Chang
Redistribution prohibited without permission

How It Works (3): Time Iteration

```python
def euler_gap(c, c_next, coh, grid, Pi, R, p: Params):
    """u'(c) - beta R E[u'(c_next(coh - c, e')) | e]: positive means c is too low."""
    ap = coh - c
    Emu = np.zeros_like(c)
    for qn in range(Pi.shape[0]):                              # tomorrow's productivity
        Emu += Pi[:, qn][None, :] * np.interp(ap, grid, c_next[:, qn]) ** (-p.sigma)
    return c ** (-p.sigma) - p.beta * R * Emu
```

```python
def time_iteration(r_net, w, transfer, grid, e, Pi, p: Params, c_init=None, tol=1e-9,
                   max_iter=5000, n_bisect=60):
    """Euler-equation iteration on a fixed grid: bisection for today's c at each node."""
    R, coh = 1.0 + r_net, cash_on_hand(r_net, w, transfer, grid, e)
    c = coh - p.a_min if c_init is None else c_init.copy()   # start: eat everything
    history = []
    for it in range(1, max_iter + 1):
        hi = coh - p.a_min                                     # a' = a_min
        binds = euler_gap(hi, c, coh, grid, Pi, R, p) >= 0.0   # the limit binds: c = hi
        lo = np.full_like(hi, 1e-10)
        for _ in range(n_bisect):                              # all nodes at once
            mid = 0.5 * (lo + hi)
            too_low = euler_gap(mid, c, coh, grid, Pi, R, p) > 0.0
            lo, hi = np.where(too_low, mid, lo), np.where(too_low, hi, mid)
        c_new = np.where(binds, coh - p.a_min, 0.5 * (lo + hi))
        history.append(float(np.abs(c_new - c).max()))
        c = c_new
        if history[-1] < tol:
            break
    return dict(ap=coh - c, c=c, sweeps=it, history=history,
                gap_evaluations_per_sweep=(n_bisect + 1) * c.size)
```

© Zhuganov Nikita · Redistribution prohibited without permission

How It Works (4): The Endogenous Grid Method

```python
def egm_step(c_next, r_net_next, r_net, w, transfer, grid, e, Pi, p: Params):
    """One backward step: tomorrow's consumption policy -> today's (c, a')."""
    R_next, R = 1.0 + r_net_next, 1.0 + r_net
    Euc = (c_next ** (-p.sigma)) @ Pi.T                      # E[u'(c') | e] at a' = grid
    c_end = (p.beta * R_next * Euc) ** (-1.0 / p.sigma)       # invert the Euler equation
    a_end = (c_end + grid[:, None] - w * e[None, :] - transfer) / R   # endogenous grid
    coh = R * grid[:, None] + w * e[None, :] + transfer       # cash on hand today
    c = np.empty_like(c_next)
    for q in range(e.size):
        c[:, q] = np.interp(grid, a_end[:, q], c_end[:, q])
        binds = grid < a_end[0, q]                            # below it the limit binds
        c[binds, q] = coh[binds, q] - p.a_min
    ap = np.clip(coh - c, p.a_min, grid[-1])
    return coh - ap, ap
```

```python
def solve_household(r_net, w, transfer, grid, e, Pi, p: Params, c_init=None,
                    tol=1e-9, max_iter=5000):
    """Stationary household problem at fixed prices. Returns (a', c, sweeps, change)."""
    coh = (1.0 + r_net) * grid[:, None] + w * e[None, :] + transfer
    c = np.maximum(coh - p.a_min if c_init is None else c_init.copy(), 1e-10)
    diff, it = np.inf, 0
    for it in range(1, max_iter + 1):
        c_new, ap = egm_step(c, r_net, r_net, w, transfer, grid, e, Pi, p)
        diff = float(np.abs(c_new - c).max())
        c = c_new
        if diff < tol:
            break
    return ap, c, it, diff
```

© Zhaoke Feng · Redistribution prohibited without permission

How It Works (5): Projection — the Polynomial and the Corner

```python
def cheb_consumption(a, theta, L, R, inc, p: Params):
    """c(a, e_q), the last axis of a indexing q: the limit binds below the corner L_q;
    above it, a Chebyshev polynomial in x, which maps [L_q, a_max] onto [-1, 1]."""
    s = np.clip((a - L) / (p.a_max - L), 0.0, 1.0)
    x = 2.0 * s ** (1.0 / p.curv) - 1.0                    # the grid's stretch (6.A1)
    T_old, T = np.ones_like(x), x
    c = theta[0] + theta[1] * x
    for k in range(2, theta.shape[0]):                     # T_k = 2x T_{k-1} - T_{k-2}
        T_old, T = T, 2.0 * x * T - T_old
        c = c + theta[k] * T
    coh = R * a + inc
    return np.where(a < L, coh - p.a_min, np.clip(c, 1e-10, coh - p.a_min))
```

```python
def corner(theta, L, R, inc, Pi, p: Params):
    """Who chooses a' = a_min exactly?  Invert the Euler equation there (EGM's Line 5)."""
    a0 = np.full(inc.size, p.a_min)
    c_next = cheb_consumption(a0, theta, L, R, inc, p)     # c(a_min, e') for every e'
    c0 = (p.beta * R * (Pi @ c_next ** (-p.sigma))) ** (-1.0 / p.sigma)
    return np.maximum((c0 + p.a_min - inc) / R, p.a_min)
```

© Zhigang Feng
Redistribution prohibited without permission

How It Works (6): Projection — Collocation, Iterated

```
def cheb_collocation(r_net, w, transfer, e, Pi, p: Params, n=20, split=False,
                     tol=1e-10, max_iter=3000, n_bisect=60):
    """n coefficients per productivity level; the Euler equation holds at n nodes."""
    R, inc = 1.0 + r_net, w * e + transfer
    z = -np.cos((2 * np.arange(n) + 1) * np.pi / (2 * n))  # the zeros of T_n (5.B2)
    s = ((z + 1.0) / 2.0) ** p.curv                        # node positions in [0, 1]
    V = np.polynomial.chebyshev.chebvander(z, n - 1)       # n x n: values = V theta
    L = np.full(e.size, p.a_min)                           # no corner yet
    a = L + (p.a_max - L) * s[:, None]                     # (n, ne) collocation nodes
    theta, history = np.linalg.solve(V, r_net * a + inc), []   # start: consume income
    for it in range(1, max_iter + 1):
        L_new = corner(theta, L, R, inc, Pi, p) if split else L
        a = L_new + (p.a_max - L_new) * s[:, None]
        coh = R * a + inc
        def gap(c):                                        # u'(c) - beta R E u'(c')
            ap = np.repeat(np.clip(coh - c, p.a_min, p.a_max)[:, :, None], e.size, 2)
            mu = cheb_consumption(ap, theta, L, R, inc, p) ** (-p.sigma)
            return c ** (-p.sigma) - p.beta * R * np.einsum("iqk,qk->iq", mu, Pi)
        hi, lo = coh - p.a_min, np.full_like(coh, 1e-10)
        binds = gap(hi) >= 0.0                             # the limit binds: c = hi
        for _ in range(n_bisect):                          # every node at once (4.A1)
            mid = 0.5 * (lo + hi)
            low = gap(mid) > 0.0
            lo, hi = np.where(low, mid, lo), np.where(low, hi, mid)
        c_new = np.where(binds, coh - p.a_min, 0.5 * (lo + hi))
        dc = float(np.abs(c_new - cheb_consumption(a, theta, L, R, inc, p)).max())
        theta, L = np.linalg.solve(V, c_new), L_new        # collocation: V theta = c
        history.append(dc)
        if dc < tol:
            break
    return dict(theta=theta, L=L, sweeps=it, history=history, nodes=a)
```

Reproduction prohibited without permission
Yongyang Feng

Takeaways

- **Part 1.** Aiyagari is three nested problems — a household, a population, a price — and every step uses a tool from Sessions 1–5. Perturbation stays on the shelf because of one kink; Chebyshev is put to the test in Part 6.
- **Part 2.** The unknown is a function on 1,400 nodes; the limit makes the Euler equation an inequality and the policy kinked, so we work on a grid.
- **Part 3.** VFI is a contraction: $\ln 10^{-8} / \ln 0.96 = 451$ sweeps, as measured. With a' on the nodes the answer is biased (capital 7% short); Howard steps cut 451 maximizations to 13.
- **Part 4.** Interpolate v and maximize by golden section: a' leaves the grid. A smooth $\hat{V}$ cuts the policy error fifty-fold.
- **Part 5.** The Euler route iterates on the policy with no maximization. Time iteration needs a root-find per node; EGM inverts and needs none.
- **Part 6.** Projection: 20 Chebyshev coefficients per level in a stretched a, the Euler equation at the zeros of T_{20}, solved by iterating K on the coefficients (Newton from a cold start fails). It rings at the kink; locating the corner lifts the worst residual from $10^{-1.01}$ to $10^{-2.16}$.
- **Part 7.** The continuous routes agree; costs differ by two orders of magnitude. A 16,000-node reference decides: smooth approximants land on $K = 4.648$, EGM's straight lines sit at 4.656. Where households live, 140 coefficients beat 1,400 node values 18-fold; EGM wins at the kink and on speed.

© Jiarong Pan • Redistribution prohibited without permission

Bridge: From a Policy to a Population

Whichever route produced it, the household's answer is a saving policy $a'(a, \varepsilon)$ on 1,400 nodes. That is all the next step reads.

Put one unit of mass on the grid, let every household follow a' and draw tomorrow's ε from Π, and repeat. Where does the mass settle — and what do we do when a' falls between two nodes?

Next: 6.A2

From a policy to a population (Young's lottery, three ways to the invariant distribution), and from a population to a price (the capital market, a bisection that must prove a sign change, five checks).

© Zhigang Feng • Redistribution prohibited without permission